

**UNIVERSIDADE ESTADUAL DE PONTA GROSSA
SETOR DE ENGENHARIAS, CIÊNCIAS AGRÁRIAS E DE
TECNOLOGIA
DEPARTAMENTO DE INFORMÁTICA**

**Carlos Eduardo Varela de Almeida
Mateus Enrick Pitura
Raissa Mayara Moreira**

**Impacto do WebAssembly no desempenho de
aplicações Rust**

**PONTA GROSSA
2026**

Carlos Eduardo Varela de Almeida

Mateus Enrick Pitura

Raissa Mayara Moreira

Impacto do WebAssembly no desempenho de aplicações Rust

Trabalho de Conclusão de Curso
apresentado ao Curso de Engenharia
de Software da Universidade Estadual
de Ponta Grossa, como requisito parcial
para obtenção do título de Bacharel em
Engenharia de Software.

Orientador: Prof. Dr. Luciano José Senger

PONTA GROSSA
2026

AGRADECIMENTOS

Expressamos nossos sinceros agradecimentos ao Prof. Dr. Luciano José Senger pela orientação dedicada ao longo deste trabalho, conduzida com atenção, paciência e compromisso acadêmico. Suas contribuições foram essenciais para o desenvolvimento e a conclusão desta pesquisa.

Aos nossos pais e amigos, manifestamos nossa profunda gratidão pelo apoio incondicional, incentivo constante e confiança depositada durante toda a trajetória acadêmica. A presença de vocês foi fundamental para a concretização deste trabalho.

RESUMO

O WebAssembly tem se consolidado como uma alternativa para execução de aplicações de alto desempenho em navegadores *web*, permitindo a utilização de linguagens compiladas além da linguagem JavaScript. Entre elas, o Rust destaca-se pela eficiência computacional e sua segurança de memória, tornando-se uma opção importante para o desenvolvimento de aplicações executadas no ambiente *web*. Neste contexto, este trabalho realiza uma análise comparativa entre a execução nativa de algoritmos implementados em Rust e a execução dos mesmos algoritmos compilados para WebAssembly. Para a realização dos experimentos, foram selecionados algoritmos representativos de diferentes classes de problemas computacionais a partir do The Computer Language Benchmarks Game. As implementações foram adaptadas para os dois ambientes de execução e avaliadas em ambiente controlado por contêineres Docker, com análise de tempo de execução e consumo de memória. Os resultados mostram que o Rust nativo foi mais rápido em todos os algoritmos avaliados, com *overhead* do WebAssembly variando de 6,7% a 129,9% conforme o algoritmo, com mediana de 18,1%. Em relação à memória, o ambiente WebAssembly manteve cerca de 210 MiB em repouso, devido ao navegador e à ferramenta de automação, o que tornou necessário o uso das métricas de *baseline* e consumo líquido para isolar o consumo dos algoritmos, que foi reduzido na maioria dos casos e maior no mandelbrot, nos dois ambientes, em valor compatível com o *buffer* do algoritmo. Conclui-se que o WebAssembly pode ser uma alternativa viável para aplicações Rust executadas no navegador, mas que o custo adicional depende fortemente da carga de trabalho, devendo ser avaliado para cada aplicação quando o desempenho for um requisito crítico. Os resultados contribuem para a compreensão dos impactos do WebAssembly em aplicações Rust e auxiliam na tomada de decisões sobre quando utilizar essa tecnologia em aplicações executadas no navegador.

Palavras-chave: Rust; WebAssembly; desempenho; benchmarking; execução nativa; Docker; Puppeteer.

LISTA DE FIGURAS

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos .	24
Figura 2 – Exemplo de uso de CPU do algoritmo <i>fasta</i> em Rust	27
Figura 3 – Fator multiplicativo do WebAssembly por algoritmo e computador . .	33

LISTA DE QUADROS

Quadro 1 – Diretrizes de viabilidade do Wasm conforme a carga	37
---	----

LISTA DE TABELAS

Tabela 1 – <i>Overhead</i> do WebAssembly por computador e consolidado	33
Tabela 2 – Tempo mediano e desvio padrão das execuções por computador . .	35
Tabela 3 – Consumo de memória do executável Rust (MiB)	35
Tabela 4 – Consumo de memória do executável WebAssembly (MiB)	35
Tabela A1 – Consumo de memória por computador (MiB)	42

LISTA DE CÓDIGOS

Código 1	–	<i>Ownership</i> e <i>borrowing</i> em Rust	14
Código 2	–	Verificação de limites e tipo <i>Option</i>	14
Código 3	–	Função Rust exportada para WebAssembly	18
Código 4	–	Chamada da função Rust a partir do JavaScript	18
Código 5	–	Memória liberada automaticamente ao fim do escopo	19
Código 6	–	Função Rust em Wasm recebendo <i>string</i> do JavaScript	20

LISTA DE ABREVIATURAS E SIGLAS

CPU	Central Processing Unit
CVE	Common Vulnerabilities and Exposures
GB	Gigabyte
HTML	HyperText Markup Language
HTTPS	Hypertext Transfer Protocol Secure
JIT	Just-In-Time
LLVM	Low Level Virtual Machine
MHz	Megahertz
MiB	Mebibyte
NVMe	Non-Volatile Memory Express
RAM	Random Access Memory
SATA	Serial AT Attachment
SSD	Solid State Drive
URL	Uniform Resource Locator
W3C	World Wide Web Consortium
Wasm	WebAssembly
WSL	Windows Subsystem for Linux

SUMÁRIO

1	INTRODUÇÃO	11
1.1	OBJETIVOS	11
1.1.1	Objetivo geral	11
1.1.2	Objetivos Específicos	11
1.2	JUSTIFICATIVA	12
2	REVISÃO DA LITERATURA	13
2.1	RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE	13
2.1.1	Segurança: o Modelo de Ownership e Borrowing	13
2.2	WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS	15
2.2.1	Arquitetura e Funcionamento	16
2.2.2	Segurança	16
2.2.3	Compilação de Rust para WebAssembly	16
2.3	ESTUDOS COMPARATIVOS DE DESEMPENHO	19
2.3.1	Rust: Desempenho sem Runtime ou Garbage Collector	19
2.3.2	WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa	20
3	MATERIAL E MÉTODOS	22
3.1	PESQUISA DA LITERATURA	22
3.2	IMPLEMENTAÇÃO PRÁTICA	22
3.2.1	Algoritmos	23
3.2.2	Configuração dos Testes	23
3.3	AMBIENTE DE TESTE	23
3.3.1	Hardwares	24
3.3.2	Softwares	25
3.4	BENCHMARKING	26
3.4.1	Coleta de métricas	26
3.4.2	Tempo de execução	28
3.4.3	Consumo de memória	28
3.4.4	Consolidação dos resultados	30
4	ANÁLISE DOS RESULTADOS	32
4.1	TEMPO DE EXECUÇÃO	32
4.2	USO DE MEMÓRIA	35
4.3	SÍNTESE COMPARATIVA	36
4.4	DIRETRIZES PARA ADOÇÃO DO WEBASSEMBLY	37
4.5	LIMITAÇÕES DO ESTUDO	37

5	CONCLUSÃO	39
	REFERÊNCIAS	40
	APÊNDICE A – CONSUMO DE MEMÓRIA POR COMPUTADOR	41

1 INTRODUÇÃO

O crescimento das aplicações *web* modernas e a necessidade cada vez maior de desempenho no lado do cliente têm tornado a escolha do ambiente de execução um aspecto importante no desenvolvimento de software. Sendo assim, o WebAssembly tem se destacado por permitir a execução de código compilado diretamente no navegador com desempenho próximo ao nativo. Paralelo a isso, a linguagem Rust vem sendo utilizada como uma das principais opções para gerar código nesse formato.

Mesmo com sua popularidade, a execução de aplicações Rust por meio do WebAssembly ainda apresenta limitações, já que o ambiente do navegador impõe restrições que não existem na execução nativa. Além disso, ainda há poucos estudos experimentais que analisem de forma prática o impacto dessa abordagem sobre o desempenho.

Diante disso, surge a questão: qual é o impacto da execução de aplicações Rust via WebAssembly no desempenho quando comparado à execução nativa, considerando diferentes tipos de problemas computacionais, e em quais situações essa abordagem pode ser considerada viável?

O objetivo deste trabalho é comparar a execução nativa de programas escritos em Rust com a execução dos mesmos programas compilados para WebAssembly no navegador, analisando o impacto em desempenho e consumo de recursos. Para isso, foram utilizados algoritmos representativos do The Computer Language Benchmarks Game (Fulgham; Gouy, 2026), adaptados para execução nos dois ambientes e avaliados sob condições controladas para coleta de métricas comparáveis.

1.1 OBJETIVOS

1.1.1 Objetivo geral

Este trabalho tem como objetivo geral realizar uma análise comparativa entre a execução de código Rust e a execução do mesmo código compilado para WebAssembly no navegador, avaliando os impactos em desempenho e consumo de recursos.

1.1.2 Objetivos Específicos

- Avaliar o consumo de recursos computacionais e tempo de execução dos algoritmos avaliados;
- Elaborar diretrizes sobre em quais cenários a execução via WebAssembly representa uma alternativa viável à execução nativa.

1.2 JUSTIFICATIVA

O avanço das aplicações *web* modernas tem impulsionado a busca por tecnologias capazes de oferecer desempenho próximo ao nativo diretamente no navegador. Nesse cenário, a combinação entre Rust e WebAssembly desponta como uma alternativa promissora, reunindo as garantias de segurança e eficiência da linguagem à portabilidade do formato binário suportado pelos principais navegadores.

Embora existam estudos comparando execução nativa e WebAssembly, estes concentram-se majoritariamente em linguagens como C e C++, deixando Rust pouco explorado nesse contexto (Jangda *et al.*, 2019). Este trabalho se justifica, portanto, tanto pelo valor técnico de produzir dados comparativos concretos sobre os dois modelos de execução quanto pela contribuição ao campo da Engenharia de Software, oferecendo subsídios para decisões arquiteturais em projetos que avaliem a viabilidade de adotar WebAssembly como ambiente de execução.

2 REVISÃO DA LITERATURA

Nesta seção são abordados os principais fundamentos e características técnicas das tecnologias Rust e WebAssembly. Inicialmente, apresenta-se a linguagem Rust, sua performance e sua proposta de garantir segurança de memória sem o uso de coletor de lixo, delegando o controle de alocação ao compilador por meio do modelo de *ownership*. Em seguida, discute-se o uso de WebAssembly como padrão binário de baixo nível suportado pelos principais navegadores, abordando sua arquitetura, modelo de execução e o processo de compilação de código Rust para esse formato.

2.1 RUST: FUNDAMENTOS, SEGURANÇA E PERFORMANCE

Rust é uma linguagem de programação de sistemas desenvolvida a partir da necessidade de conciliar alto desempenho e segurança no desenvolvimento de software. Desde que a Mozilla começou a patrocinar seu desenvolvimento em 2010, a linguagem foi ganhando espaço na comunidade devido à forma como ela lida com dois aspectos historicamente desafiadores no desenvolvimento de software: o desempenho e a segurança de memória.

2.1.1 Segurança: o Modelo de Ownership e Borrowing

Embora o desempenho da linguagem Rust seja expressivo, é no âmbito da segurança que suas principais características se destacam. Segundo o Microsoft Security Response Center, cerca de 70% das vulnerabilidades que recebem CVE da Microsoft a cada ano têm origem em falhas de segurança de memória (Thomas, 2019). O Rust enfrenta esse problema por meio de um mecanismo denominado *ownership* (propriedade).

O princípio estabelece que cada valor possui um único proprietário, responsável pelo gerenciamento de seu ciclo de vida. Quando o proprietário deixa de estar em escopo, o valor correspondente é automaticamente desalocado. Isso impede, por exemplo, que o programa use um dado que já foi descartado, erro conhecido como *use after free*, comum em C e frequentemente explorado por atacantes. Quando é preciso acessar um valor sem tomar sua propriedade, usa-se o *borrowing* (empréstimo), que segue uma regra: ou várias partes do código leem o dado ao mesmo tempo (referências imutáveis), ou uma única parte pode modificá-lo (referência mutável), nunca as duas coisas ao mesmo tempo (Bugden; Alahmar, 2022).

O Código 1 reúne os dois conceitos.

Código 1 – *Ownership* e *borrowing* em Rust

```
fn main() {  
    // Ownership: o valor é transferido (move) para 'linguagem'.  
    let nome = String::from("Rust");  
    let linguagem = nome;  
    // println!("{}", nome); // erro: 'nome' não é mais o dono  
  
    // Borrowing: empréstimo imutável seguido de empréstimo mutável.  
    let mut valores = String::from("Rust");  
    let leitura = &valores;  
    // let alteracao = &mut valores; // erro: 'leitura' ainda está ativa  
    println!("Conteúdo: {}", leitura); // último uso de 'leitura'  
  
    let alteracao = &mut valores; // permitido: não há mais conflito  
    alteracao.push_str(" é seguro");  
    println!("{}", linguagem, alteracao);  
}
```

Fonte: Os autores

Na primeira parte, após a transferência, `nome` deixa de ser válida e qualquer tentativa de usá-la gera erro de compilação. Na segunda, o compilador recusa a referência mutável enquanto a imutável ainda será usada, mas a aceita logo depois do último uso de `leitura`, pois os empréstimos deixam de se sobrepor. Essa verificação impede as condições de corrida (*data races*), em que diferentes partes do programa acessam e modificam a mesma região de memória ao mesmo tempo sem sincronização (Bugden; Alahmar, 2022).

Além disso, fora de blocos marcados como *unsafe*, Rust não permite manipular endereços de memória diretamente, e todo acesso a vetores tem os limites verificados: um índice inexistente encerra o programa de forma controlada (*panic*), em vez de ler dados que não deveria (Bugden; Alahmar, 2022). A linguagem também elimina o ponteiro nulo, substituindo-o pelo tipo *Option*, que obriga o programador a tratar explicitamente a ausência de um valor (Bugden; Alahmar, 2022). O Código 2 mostra os dois mecanismos juntos.

Código 2 – Verificação de limites e tipo *Option*

```
fn main() {  
    let numeros = vec![10, 20, 30]; // índices válidos: 0, 1 e 2  
  
    // println!("{}", numeros[5]); // panic: index out of bounds
```

```
// 'get' retorna Option: Some(valor) ou None.  
match numeros.get(5) {  
    Some(n) => println!("Valor: {}", n),  
    None => println!("Posição inexistente"),  
}  
}
```

Fonte: Os autores

O acesso direto `numeros[5]` compila, mas é interrompido em tempo de execução. Já o método `get` devolve `None`, e o `match` obriga o código a tratar esse caso, sem pressupor que o valor exista.

O que torna tudo isso notável é que a maior parte dessas garantias é verificada em tempo de compilação: as regras de *ownership*, *borrowing* e o tratamento de *Option* são checados antes de o programa existir, enquanto o acesso fora dos limites é contido durante a execução. Erros que em outras linguagens só aparecem em produção, às vezes explorados por um atacante, em Rust são barrados muito antes (Bugden; Alahmar, 2022).

2.2 WEBASSEMBLY: AMBIENTE E CARACTERÍSTICAS

Se Rust resolve o problema de escrever código seguro e eficiente, o WebAssembly resolve um problema diferente: como fazer esse código ser executado dentro de um navegador com desempenho próximo ao nativo. É justamente essa combinação que torna as duas tecnologias complementares e justifica o foco deste trabalho.

Por muito tempo, o JavaScript foi a única linguagem executada pelos navegadores. Isso funcionou para páginas simples, mas com o crescimento de aplicações mais pesadas, editores de imagem, jogos, processamento de vídeo, suas limitações ficaram evidentes. Foi para resolver esse problema que Google, Microsoft, Mozilla e Apple se uniram para criar o WebAssembly, cuja primeira versão foi consolidada em 2017 (Haas *et al.*, 2018) e posteriormente estabelecido como padrão oficial do W3C em dezembro 2019 (World Wide Web Consortium (W3C), 2019).

WebAssembly é um formato binário compacto de baixo nível que executa com desempenho próximo ao nativo, servindo como alvo de compilação para linguagens como Rust, C e C++. Ele não foi criado para substituir o JavaScript, mas para trabalhar ao lado dele: enquanto o JavaScript cuida da interface e das interações, o Wasm assume as partes que exigem mais processamento (MDN Web Docs, 2026).

2.2.1 Arquitetura e Funcionamento

Um programa WebAssembly é organizado em módulos isolados, onde o código de um módulo não acessa dados de outro a menos que isso seja explicitamente permitido. O armazenamento de dados acontece em uma estrutura chamada memória linear, um vetor de bytes completamente separado do código executável e das estruturas internas do motor de execução. Isso garante que mesmo um programa com falha não consiga corromper o ambiente ao redor (Moric; Branstett; Petrunic, 2022).

Esse isolamento tem uma consequência prática importante: módulos de origens diferentes podem executar no mesmo processo sem interferir uns nos outros, o que viabiliza o uso do Wasm em ambientes como servidores na nuvem e dispositivos embarcados, não apenas em navegadores (Moric; Branstett; Petrunic, 2022).

2.2.2 Segurança

A segurança faz parte da especificação do WebAssembly desde o início, e não foi adicionada depois como um recurso extra. Um módulo Wasm não possui acesso a nenhum recurso do sistema, arquivos, redes ou dispositivos, a menos que o ambiente hospedeiro conceda esse acesso de forma explícita. Além disso, o fluxo de execução é validado antes de o programa executar por meio de um mecanismo chamado integridade do fluxo de controle (*Control-Flow Integrity*), que impede que código malicioso redirecione a execução para posições não autorizadas na memória (Moric; Branstett; Petrunic, 2022).

Como os métodos do WebAssembly são tipados, o uso incorreto de tipos, como passar um valor do tipo errado para uma função, não é possível, o que reforça ainda mais a confiabilidade do ambiente de execução. Quando combinado com as garantias de segurança de memória do Rust, esse modelo cria uma camada dupla de proteção: o compilador Rust elimina os erros antes da geração do binário, e o WebAssembly garante que o binário gerado rode dentro de limites seguros (Moric; Branstett; Petrunic, 2022).

2.2.3 Compilação de Rust para WebAssembly

O processo de transformar código Rust em um módulo WebAssembly é direto e bem suportado pelo ecossistema da linguagem. O fluxo básico envolve três etapas: o código é escrito em Rust normalmente, compilado para o formato binário WebAssembly (.wasm) por meio de ferramentas como o wasm-pack, e então carregado e executado pelo ambiente de destino, seja um navegador, um servidor ou outro *runtime* compatível.

O `wasm-pack` é a ferramenta principal para esse processo no ecossistema Rust. Ela cuida de compilar o código, gerar os arquivos de integração com JavaScript e empacotar tudo em um formato pronto para ser publicado ou importado por uma aplicação *web*. Por baixo, o compilador Rust utiliza o *backend* LLVM para gerar o binário `.wasm`, o mesmo *backend* responsável pelas otimizações que tornam o código Rust tão eficiente em ambiente nativo (Moric; Branstett; Petrunic, 2022).

Para exemplificar esse processo, o Código 3 apresenta uma função implementada em Rust e preparada para ser exportada para WebAssembly. A função calcula o *n*-ésimo termo da sequência de Fibonacci e utiliza o atributo `#[wasm_bindgen]` para permitir sua integração com JavaScript.

Código 3 – Função Rust exportada para WebAssembly

```
use wasm_bindgen::prelude::*;

#[wasm_bindgen]
pub fn fibonacci(n: u32) -> u64 {
    match n {
        0 => 0,
        1 => 1,
        _ => {
            let (mut a, mut b) = (0u64, 1u64);
            for _ in 2..=n {
                let temp = a + b;
                a = b;
                b = temp;
            }
            b
        }
    }
}
```

Fonte: Os autores

A macro `#[wasm_bindgen]` gera os *bindings* necessários para que a função apresentada no Código 3 possa ser chamada a partir do JavaScript, conforme demonstrado no Código 4.

Código 4 – Chamada da função Rust a partir do JavaScript

```
import init, { fibonacci } from "./pkg/meu_projeto.js";

async function executar() {
    await init(); // carrega e instancia o módulo .wasm
    const resultado = fibonacci(30);
    console.log(resultado);
}

executar();
```

Fonte: Os autores

Nesse caso, o JavaScript inicializa o módulo WebAssembly e, em seguida, chama a função `fibonacci`, passando o valor 30 como parâmetro. O resultado

retornado pela função Rust é então exibido no console do navegador.

Vale destacar que nem todo código Rust compila para WebAssembly sem adaptações. Funcionalidades que dependem diretamente do sistema operacional, como acesso a arquivos ou criação de *threads*, não estão disponíveis no ambiente do navegador por padrão.

2.3 ESTUDOS COMPARATIVOS DE DESEMPENHO

Compreender o impacto do WebAssembly no desempenho de aplicações Rust exige fazer duas análises: primeiro, o que torna o Rust eficiente em ambiente nativo; segundo, o que acontece com esse desempenho quando o mesmo código é compilado para WebAssembly e executado dentro do navegador. As subseções a seguir percorrem essas duas camadas, formando a base comparativa que orienta a investigação empírica deste trabalho.

2.3.1 Rust: Desempenho sem Runtime ou Garbage Collector

Quando um programa é executado, ele precisa reservar espaço na memória para guardar seus dados, e também precisa liberar esse espaço quando os dados não são mais necessários. Linguagens como Java e Python fazem isso automaticamente por meio de um mecanismo chamado coletor de lixo (*garbage collector*), que fica em segundo plano verificando o que pode ser descartado. Isso é conveniente, mas tem um custo: o coletor consome processamento, ocupa memória extra e pode pausar o programa em momentos imprevisíveis para fazer sua limpeza (Bugden; Alahmar, 2022).

Rust não tem coletor de lixo. Em vez disso, o compilador analisa o código antes de o programa ser executado e já determina exatamente quando cada dado deve ser criado e quando deve ser descartado. Quando uma variável chega ao fim do bloco em que foi criada, sua memória é liberada automaticamente, sem nenhuma instrução extra do programador e sem nenhum custo em tempo de execução.

O Código 5 ilustra esse comportamento por meio de uma variável do tipo `String`. Ao atingir o final do escopo em que foi declarada, a variável deixa de existir e a memória associada ao seu valor é liberada automaticamente.

Código 5 – Memória liberada automaticamente ao fim do escopo

```
fn main() {  
    // 'mensagem' é criada aqui e pertence a este escopo  
    let mensagem = String::from("olá, Mundo!");  
    println!("{}", mensagem);  
    // Ao chegar aqui, 'mensagem' sai de escopo
```

```
// e sua memória é liberada automaticamente pelo compilador.  
// Nenhuma instrução manual é necessária.  
}
```

Fonte: Os autores

O exemplo demonstra, portanto, que a liberação da memória ocorre automaticamente ao final do escopo, sem a necessidade de uma chamada explícita para liberar o recurso.

Esse modelo elimina tanto o trabalho manual de liberar memória, que em C frequentemente gera erros, quanto a sobrecarga do coletor de lixo. O resultado é um programa que roda com eficiência comparável à de C e C++, mas sem exigir que o programador gerencie a memória manualmente (Bugden; Alahmar, 2022).

É esse nível de eficiência nativa que serve de referência para a comparação com o WebAssembly. Quando o mesmo código Rust é compilado para o formato `.wasm` e executado no navegador, parte dessas garantias de desempenho precisa ser reexaminada, pois o ambiente de execução impõe restrições e custos que não existem na compilação nativa.

2.3.2 WebAssembly: Desempenho Frente ao JavaScript e à Execução Nativa

Dentro do ambiente do navegador, o WebAssembly já representa um avanço considerável em relação ao JavaScript. Em tarefas computacionais intensas, o WebAssembly supera o `asm.js` (subconjunto otimizável do JavaScript e seu antecessor mais próximo), sendo em média 1,3 vezes mais rápido (Jangda *et al.*, 2019). No entanto, há uma ressalva importante: transferir dados complexos para dentro da memória do módulo tem um custo. Para passar uma *string* ao Wasm, por exemplo, é necessário codificá-la, localizar espaço livre na memória linear e escrevê-la *byte a byte*, um processo que pode anular o ganho de desempenho em operações simples com texto. Quando o dado já está na memória ou o algoritmo é suficientemente complexo, o Wasm volta a superar o JavaScript por ampla margem (Moric; Branstett; Petrunic, 2022).

Para ilustrar essa interação entre JavaScript e WebAssembly, o Código 6 apresenta uma função Rust que recebe uma string como parâmetro e realiza a contagem de suas vogais. O exemplo evidencia o uso de dados textuais na comunicação entre o código JavaScript e o módulo WebAssembly.

Código 6 – Função Rust em Wasm recebendo *string* do JavaScript

```
use wasm_bindgen::prelude::*;
```

```
#[wasm_bindgen]
pub fn contar_vogais(texto: &str) -> u32 {
    texto
        .chars()
        .filter(|c| "aeiouAEIOU".contains(*c))
        .count() as u32
}
```

Fonte: Os autores

Contudo, a comparação mais relevante para este trabalho não é entre WebAssembly e JavaScript, mas entre WebAssembly e a execução nativa do próprio Rust. Nesse confronto, o cenário é diferente. Estudos mostram que aplicações compiladas para WebAssembly são executadas, em média, entre 45% e 55% mais devagar do que o equivalente nativo, a depender do navegador utilizado (Jangda *et al.*, 2019). Esse é exatamente o *gap* que este trabalho busca caracterizar no contexto específico de aplicações Rust, contribuindo com dados concretos para orientar decisões sobre quando a portabilidade do Wasm justifica a perda de desempenho em relação à execução nativa.

3 MATERIAL E MÉTODOS

A abordagem deste trabalho é quantitativa, buscando mensurar o desempenho e o consumo de recursos de diferentes algoritmos. Para isso, foram adotadas as etapas de pesquisa da literatura, implementação prática e análise comparativa.

3.1 PESQUISA DA LITERATURA

Foram reunidas informações sobre Rust e WebAssembly na pesquisa da literatura, destacando seu contexto histórico, características técnicas e aplicabilidade.

Em relação ao Rust, foram abordados tópicos como o sistema de propriedade (*ownership*) e o modelo de empréstimo de valores (*borrowing*). Já para o WebAssembly, destacaram-se aspectos como o gerenciamento de memória, a segurança proporcionada pela restrição de permissões e sua integração com Rust por meio da portabilidade de código utilizando ferramentas como o *wasm-pack*.

3.2 IMPLEMENTAÇÃO PRÁTICA

Na etapa de implementação, foram utilizados cinco algoritmos distintos em Rust, cada um explora diferentes características de execução, como operações de inteiros e de pontos flutuantes, manipulação de *arrays*, laços de repetição, manipulação de *bits*, geração de número pseudo aleatórios e saída de dados. Os algoritmos foram obtidos do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026) e sofreram adaptações mínimas para serem executadas no ambiente de teste. Buscou-se utilizar o mínimo possível de bibliotecas externas, limitando-se à biblioteca Rayon, utilizada nos algoritmos *fannkuch-redux*, *mandelbrot* e *spectral-norm*, para processamento paralelo e às bibliotecas nativas da linguagem, a fim de reduzir interferências externas nos resultados de desempenho.

No contexto do Rust, foi desenvolvido um código de entrada responsável por receber, por meio de parâmetros posicionais, o algoritmo a ser executado e a quantidade de execuções. Esse código também realiza a coleta do tempo de início e término da execução, mantendo os algoritmos isolados do processo de metrificação.

Para o WebAssembly, os algoritmos implementados em Rust foram portados utilizando a ferramenta *wasm-pack*. Além disso, foi criado um arquivo de entrada em HTML para carregar uma página contendo código JavaScript capaz de executar os algoritmos portados. Essa página também aceita parâmetros via URL para definir o algoritmo e a quantidade de execuções. A execução ocorre em um navegador sem interface gráfica (*headless*), utilizando a biblioteca Puppeteer de JavaScript.

Ambas as versões foram compiladas utilizando o perfil de compilação `release` padrão. Não foram especificadas flags adicionais de compilação, de modo que foram mantidas as configurações de otimização e demais opções predefinidas nesse perfil.

3.2.1 Algoritmos

Os seguintes algoritmos foram utilizados na implementação prática:

- `fannkuch-redux`: foca em operações com inteiros e manipulação de *arrays*;
- `n-body`: realiza operações de ponto flutuante;
- `spectral-norm`: concentra-se em laços de repetição;
- `mandelbrot`: faz a manipulação de *bits*;
- `fasta`: avalia a geração de números pseudoaleatórios e a formatação das sequências geradas em blocos de memória.

3.2.2 Configuração dos Testes

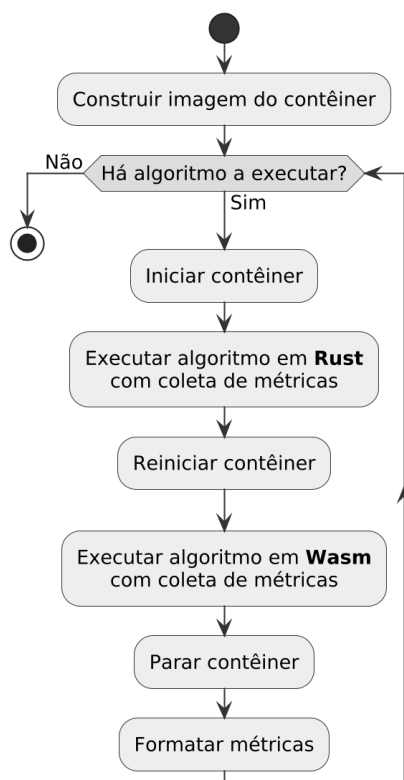
Cada algoritmo foi executado 10 vezes em sequência. Antes da coleta de métricas de cada algoritmo, ele foi executado 1 vez, com o objetivo de diminuir a influência do JIT e do *cache*. Os algoritmos foram executados com uma única *thread* nos dois ambientes, limitando a biblioteca Rayon por meio da variável de ambiente `RAYON_NUM_THREADS`, pois o processamento paralelo no WebAssembly exige a biblioteca `wasm-bindgen-rayon`, que não pôde ser compilada no ambiente de testes. Dessa forma, os dois ambientes executaram a mesma carga, com o mesmo número de *threads*. A parametrização dos algoritmos foi a mesma usada nos testes de desempenho do site The Computer Language Benchmarks Game (Fulgham; Gouy, 2026). O fluxo de execução dos testes e coleta de métricas dos algoritmos é ilustrado na Figura 1.

3.3 AMBIENTE DE TESTE

Com o objetivo de isolar o ambiente de execução, foram utilizados contêineres por meio do Docker, com quantidades controladas de CPUs e memória RAM, limitadas a dois núcleos de processamento e 2 GB de memória, visando aumentar a reprodutibilidade dos experimentos.

Os testes foram realizados em 3 computadores diferentes. Em cada computador, foram calculados a mediana, a média e o desvio padrão dos tempos das

Figura 1 – Fluxo de execução dos testes e coleta de métricas dos algoritmos



Fonte: Os autores

execuções de cada algoritmo, e os resultados dos computadores foram posteriormente consolidados, conforme descrito na Seção 3.4.4. Além disso, foram padronizadas as versões das tecnologias utilizadas em todos os computadores. A seguir, são apresentados os hardwares e os softwares utilizados.

3.3.1 Hardwares

Computador 1:

- Sistema operacional: Ubuntu 24.04 (Noble Numbat);
- Kernel: Linux 6.17.0-35-generic;
- Processador: Intel i7 1165G7, 4 núcleos / 8 threads;
- Memória RAM: 16 GB LPDDR4 (8x2 GB) 4267 MHz;
- Armazenamento: SSD NVMe de 1 TB Micron MTFDHBA1T0QFD.

Computador 2:

- Sistema operacional: Ubuntu 24.04 (Noble Numbat) via WSL 2;
- Kernel: Linux 6.6.87.2-microsoft-standard-WSL2;
- Processador: Intel Core i5-10300H, 4 núcleos / 8 threads;
- Memória RAM: 8 GB DDR4 (8x1 GB) 3200 MHz;
- Armazenamento: SSD SATA de 500 GB KINGSTON SA400S37.

Computador 3:

- Sistema operacional: Ubuntu 22.04 (Jammy Jellyfish) via WSL 2;
- Kernel: Linux 6.6.114.1-microsoft-standard-WSL2;
- Processador: AMD Ryzen 5 5600x, 6 núcleos / 12 threads;
- Memória RAM: 32 GB DDR4 (16x2 GB) 3200 MHz;
- Armazenamento: SSD NVMe de 500 GB KINGSTON SNVS500G.

3.3.2 Softwares

- Docker: versão 29.5.3;
- Docker Compose: versão v5.1.4;
- Rust: versão 1.95.0;
- wasm-pack: 0.14.0;
- wasm-bindgen: 0.2.121;
- Rayon: 1.8;
- Puppeteer: 24.43.1;
- Chrome for Testing (modo *headless*): 148.0.7778.97;
- Node.js: v24;
- npm: 10.9.4.

3.4 BENCHMARKING

Inicialmente, foi considerada a utilização de bibliotecas como Criterion e Hyperfine para a realização do *benchmarking*. Entretanto, para simplificar a implementação, foram desenvolvidos *scripts* responsáveis por automatizar a coleta de métricas dentro do contêiner Docker, além de código em Python para formatação e processamento dos dados coletados.

Essa abordagem também permitiu alterar parâmetros dos testes por meio de arquivos de configuração e realizar ajustes finos, buscando reduzir ao máximo interferências externas nos resultados.

3.4.1 Coleta de métricas

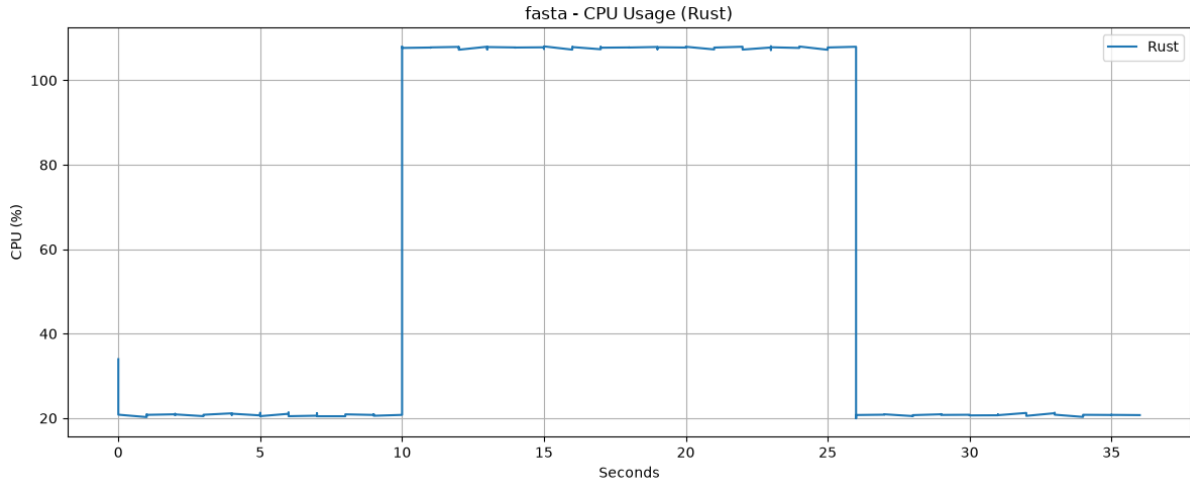
Durante a avaliação, foram coletadas as métricas de tempo de execução, uso de memória ao longo do tempo e uso de CPU ao longo do tempo. Os *scripts* desenvolvidos para a automação dos testes, a coleta dessas métricas e o código-fonte dos algoritmos utilizados estão disponíveis publicamente em um repositório no GitHub¹. Para reduzir a interferência do consumo de recursos em estado de espera (*standby*) pelo contêiner e, no caso do Wasm, pelo Puppeteer, além da sobrecarga de carregamento da página *web*, foi adotado um tempo de 10 segundos em estado ocioso (*idle*) antes e depois de cada execução. Além disso, o contêiner era reiniciado antes de cada execução para remover recursos residuais.

A Figura 2 apresenta um exemplo do uso de CPU durante a execução do algoritmo *fasta* em Rust, com 10 segundos em estado ocioso (*idle*) antes e depois da execução.

O tempo de execução foi medido pelo próprio código de entrada de cada ambiente, utilizando as funções de medição de tempo de cada linguagem: a estrutura *Instant* da biblioteca padrão do Rust, no ambiente nativo, e a função *performance.now()* do JavaScript, no ambiente WebAssembly. Essas funções medem intervalos de tempo sem sofrer influência de ajustes no relógio do sistema. Após a execução de aquecimento, foi registrado o tempo de cada uma das $N = 10$ execuções consecutivas, $t_1, t_2, \dots, t_N$, em milissegundos. O tempo de execução de cada algoritmo, em cada ambiente e computador, corresponde à mediana desses valores, conforme a Equação (3.1), por ser menos sensível a execuções atípicas do que a média:

$$\tilde{t} = \text{mediana}\{t_1, t_2, \dots, t_N\} \quad (3.1)$$

¹ Disponível em: <<https://github.com/cmr-tcc/rust-wasm/tree/0234cc3>>, versão (*commit*) 0234cc3.

Figura 2 – Exemplo de uso de CPU do algoritmo *fasta* em Rust

Fonte: Os autores

A dispersão entre as execuções foi medida pelo desvio padrão amostral, conforme a Equação (3.2):

$$s = \sqrt{\frac{1}{N-1} \sum_{i=1}^N (t_i - \bar{t})^2} \quad (3.2)$$

em que $\bar{t}$ representa a média aritmética das N execuções.

O uso de memória e de CPU foi coletado por um *script* executado dentro do próprio contêiner, que realizava leituras periódicas dos contadores do *cgroup* do Linux, mecanismo do núcleo do sistema operacional responsável por contabilizar os recursos utilizados pelo contêiner. Entre as leituras, foi utilizado um intervalo de espera de 250 milissegundos, o que, somado ao tempo de leitura das métricas, resultou em uma taxa mediana entre 2,8 e 3,2 amostras por segundo nos três computadores, calculada pela razão entre o número de amostras e a duração de cada coleta. O *script* também registrava o consumo individual de cada processo, porém, na análise, foram consideradas apenas as amostras referentes ao contêiner como um todo.

A utilização de CPU na i -ésima amostra foi calculada pela Equação (3.3):

$$cpu_i = \frac{u_i - u_{i-1}}{\tau_i - \tau_{i-1}} \times 100 \quad (3.3)$$

em que u_i representa o tempo acumulado de CPU consumido pelo contêiner, em microssegundos, e τ_i o instante da leitura, também em microssegundos. Como o contêiner dispunha de dois núcleos, o valor pode ultrapassar 100%.

Embora coletada, a utilização de CPU não foi analisada como resultado comparativo entre os ambientes, sendo empregada apenas para separar os períodos de repouso e de execução na análise de memória, descrita na Seção 3.4.3. O consumo de memória na i -ésima amostra foi calculado pela Equação (3.4):

$$m_i = \frac{M_i - C_i}{2^{20}} \quad (3.4)$$

em que M_i representa a memória total contabilizada para o contêiner, em *bytes*, e C_i a parcela de *cache* de arquivos inativa, que pode ser liberada pelo sistema operacional a qualquer momento e, portanto, não representa consumo efetivo. A divisão por 2^{20} converte o valor para MiB.

3.4.2 Tempo de execução

Para a análise de desempenho temporal, a execução nativa do Rust foi adotada como referência, representando o limite superior. Em relação a essa referência, o *overhead* percentual do WebAssembly foi calculado pela Equação (3.5), e o fator multiplicativo, que indica quantas vezes a execução em WebAssembly foi mais lenta que a nativa, pela Equação (3.6):

$$\text{overhead} (\%) = \left(\frac{t_{\text{wasm}}}{t_{\text{rust}}} - 1 \right) \times 100 \quad (3.5)$$

$$\text{fator} = \frac{t_{\text{wasm}}}{t_{\text{rust}}} = 1 + \frac{\text{overhead} (\%)}{100} \quad (3.6)$$

em que t_{rust} e t_{wasm} representam os tempos medianos de execução (Equação (3.1)), em milissegundos, de cada ambiente em um mesmo computador.

Como os computadores possuem configurações de *hardware* distintas, os tempos absolutos de máquinas diferentes não foram combinados. O fator multiplicativo f_j de cada algoritmo foi calculado em cada computador j , a partir dos seus próprios tempos medianos (Equação (3.6)), e os fatores dos $K = 3$ computadores foram consolidados pela média geométrica, forma adequada para resumir razões de desempenho (Fleming; Wallace, 1986), conforme a Equação (3.7):

$$\bar{f} = \left(\prod_{j=1}^K f_j \right)^{1/K} \quad (3.7)$$

O *overhead* consolidado de cada algoritmo foi obtido a partir de $\bar{f}$, pela relação da Equação (3.6), ou seja, $(\bar{f} - 1) \times 100$.

3.4.3 Consumo de memória

Para a análise de memória, foram derivadas três métricas a partir das amostras coletadas em cada execução: *baseline*, pico e consumo líquido. Para isso, foi necessário separar o período ocioso do período de execução ativa. Como os algoritmos foram executados com uma única *thread*, a execução ativa

ocupa integralmente um núcleo de processamento, o que corresponde a 100% na Equação (3.3), elevando a utilização de CPU do contêiner acima desse valor. Não foi adotado um limiar inferior para identificar a execução porque, mesmo sem nenhum algoritmo em execução, a utilização de CPU do contêiner no período ocioso anterior à execução foi em média de 21,5% no Rust e 46,4% no Wasm, chegando a 95,8%.

Para evitar que picos momentâneos de CPU, como os causados pela inicialização do navegador, fossem confundidos com a execução do algoritmo, foi definido o parâmetro k , que corresponde ao número mínimo de amostras consecutivas com utilização de CPU acima de 100% para que um trecho seja considerado parte da execução. Foi adotado $k = 8$, o que corresponde a cerca de 2 a 3 segundos de coleta. O período de execução inicia na primeira amostra do primeiro trecho que atende a esse critério, de índice a , e termina na última amostra do último trecho, de índice b . O conjunto de execução E reúne as amostras desse período com utilização de CPU acima de 100%, conforme a Equação (3.8):

$$E = \{ i \mid a \leq i \leq b \wedge cpu_i > 100\% \} \quad (3.8)$$

De forma análoga, foi definido o parâmetro n , que corresponde ao número de amostras de repouso utilizadas no cálculo do *baseline*. O conjunto ocioso O reúne as n últimas amostras anteriores ao início da execução (a) com utilização de CPU igual ou inferior a 100%, conforme a Equação (3.9), considerando-se apenas os n maiores índices que satisfazem a condição. Foi adotado $n = 10$, correspondendo ao final dos 10 segundos de repouso que antecedem cada execução:

$$O = \{ i \mid i < a \wedge cpu_i \leq 100\% \} \quad (3.9)$$

O consumo de *baseline* (Equação (3.10)) corresponde ao mínimo entre as amostras do conjunto ocioso O , representando o consumo do contêiner em repouso. O consumo de pico (Equação (3.11)) corresponde ao máximo entre as amostras do conjunto de execução E , representando o consumo durante a execução ativa. O consumo líquido é dado pela diferença entre pico e *baseline* (Equação (3.12)):

$$baseline = \min\{ m_i \mid i \in O \} \quad (3.10)$$

$$pico = \max\{ m_i \mid i \in E \} \quad (3.11)$$

$$líquido = pico - baseline \quad (3.12)$$

Essa distinção é necessária porque o contêiner Wasm mantém o Chrome Headless e o Puppeteer em execução permanente, elevando o *baseline* a cerca de 210 MiB independentemente do algoritmo executado.

Para verificar se os resultados dependiam da escolha de k e n , as métricas de memória foram recalculadas para todos os pares formados por um valor de k entre 8 e 20 (13 valores) e um valor de n entre 3 e 17 (15 valores), totalizando 195 pares de parâmetros. Em todos eles, o consumo líquido médio variou no máximo 0,2 MiB em relação aos valores adotados ($k = 8$ e $n = 10$). Valores de k menores que 8 ou de n maiores que 17 passam a incluir o período de carregamento do navegador, no qual a memória ainda está em crescimento, e por isso não foram considerados.

3.4.4 Consolidação dos resultados

Os procedimentos anteriores foram aplicados separadamente aos dados de cada computador. Para o tempo de execução, os fatores de cada computador foram consolidados pela média geométrica (Equação (3.7)), conforme descrito na Seção 3.4.2. As métricas de memória, que não dependem da velocidade de cada máquina, uma vez que os contêineres possuíam os mesmos limites de recursos, foram consolidadas pela média aritmética entre os computadores, conforme a Equação (3.13):

$$\bar{x} = \frac{1}{K} \sum_{j=1}^K x_j \quad (3.13)$$

em que x_j representa o valor de *baseline*, pico ou consumo líquido (Equações (3.10) a (3.12)) obtido no j -ésimo computador. Os tempos medianos e os desvios padrão de cada computador são apresentados no Capítulo 4, e as métricas de memória de cada computador, no Apêndice A.

Por fim, para sintetizar o comportamento geral, foram calculadas a média e a mediana do *overhead* entre os $A = 5$ algoritmos avaliados. A média foi obtida pela Equação (3.14), e a mediana corresponde ao valor central do conjunto de *overheads* ordenado de forma crescente, conforme a Equação (3.15):

$$\bar{o} = \frac{1}{A} \sum_{k=1}^A o_k \quad (3.14)$$

$$\tilde{o} = o_{(\frac{A+1}{2})} \quad (3.15)$$

em que o_k representa o *overhead* do k -ésimo algoritmo e $o_{(j)}$ o j -ésimo menor valor entre eles. Como A é ímpar, a mediana corresponde ao terceiro menor *overhead*. A mediana foi incluída por ser menos sensível a valores extremos do que a média. Os resultados de tempo, desvio padrão, *overhead* e memória foram apresentados com uma casa decimal, e o fator multiplicativo com três, de modo a preservar a correspondência entre as duas métricas, dada pela Equação (3.6).

O processamento dos dados foi realizado por meio de *scripts* em Python desenvolvidos pelos autores.

4 ANÁLISE DOS RESULTADOS

As métricas coletadas foram analisadas com base no levantamento teórico realizado, buscando avaliar os aspectos de desempenho de maneira objetiva e fundamentada em cada cenário analisado. Para o tempo de execução, os valores consolidados correspondem à média geométrica dos fatores obtidos em cada um dos três computadores descritos na metodologia. Para o consumo de memória, correspondem à média aritmética entre eles.

Para o tempo de execução, foi comparado o resultado entre os dois ambientes, expressando a diferença como *overhead* percentual e fator multiplicativo.

Para o consumo de memória, foram aplicadas as métricas de *baseline*, pico e consumo líquido descritas anteriormente, a fim de isolar o consumo efetivo do algoritmo do *overhead* fixo do ambiente de execução Wasm.

A utilização de CPU, embora coletada, foi empregada apenas para identificar os períodos de execução, pelos motivos descritos na Seção 3.4.3. A média dessa métrica permaneceu praticamente constante entre os algoritmos em cada computador e ambiente, como no computador 1, em que variou de 107,3% a 108,0% no Rust e de 123,9% a 124,7% no Wasm, de modo que ela não diferenciou os algoritmos avaliados.

4.1 TEMPO DE EXECUÇÃO

A Tabela 1 apresenta o *overhead* do WebAssembly em cada computador, calculado a partir dos tempos medianos de cada máquina (Equações (3.1) e (3.5)), e o *overhead* e o fator multiplicativo consolidados pela média geométrica (Equação (3.7)). Nessa comparação, o Rust nativo é a referência, limite superior de desempenho, de modo que o *overhead* e o fator multiplicativo medem o desempenho do WebAssembly em relação a esse limite. Os tempos medianos e os desvios padrão de cada computador são apresentados na Tabela 2. Os resultados mostram que o *overhead* consolidado variou de 6,7% a 129,9%, indicando diferenças distintas entre os algoritmos avaliados. Em todos os casos e em todos os computadores, o WebAssembly apresentou tempo de execução maior que o Rust nativo.

Os menores *overheads* foram observados em *fannkuch-redux* e *spectral-norm*, com 6,7% e 14,0%, respectivamente. Em contraste, *n-body* e *mandelbrot* apresentaram as maiores diferenças, com 69,7% e 129,9%. A Figura 3 apresenta o fator multiplicativo de cada algoritmo, com uma coluna para cada computador e um traço horizontal para o fator consolidado, permitindo visualizar tanto a diferença entre os algoritmos quanto a variação entre as máquinas.

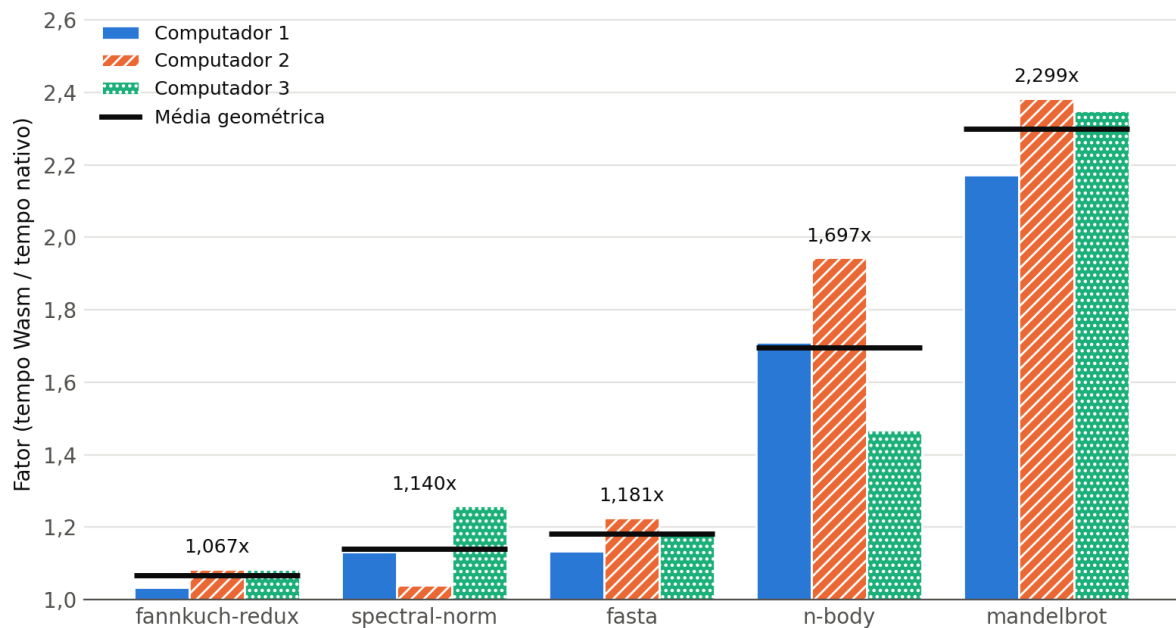
O algoritmo *fannkuch-redux* registrou o menor *overhead*, correspondente a 6,7%, ou fator 1,067x. Trata-se de um algoritmo baseado em permutações de vetores

Tabela 1 – *Overhead* do WebAssembly por computador e consolidado

Algoritmo	Overhead por computador (%)			Overhead (%)	Fator
	1	2	3		
fannkuch-redux	3,3	8,4	8,4	6,7	1,067x
spectral-norm	13,3	4,0	25,9	14,0	1,140x
fasta	13,3	22,6	18,6	18,1	1,181x
n-body	71,0	94,4	46,9	69,7	1,697x
mandelbrot	117,1	138,3	135,0	129,9	2,299x

Fonte: Os autores

Figura 3 – Fator multiplicativo do WebAssembly por algoritmo e computador



Fonte: Os autores

de inteiros de tamanho fixo, com operações predominantemente inteiras.

O *spectral-norm* apresentou *overhead* de 14,0%. O algoritmo realiza operações de ponto flutuante de dupla precisão em laços sobre vetores.

O algoritmo *fasta* obteve *overhead* de 18,1%. Sua execução envolve a geração de números pseudoaleatórios, consultas a tabelas de probabilidade acumulada e a formatação das sequências em linhas de 60 caracteres. As sequências geradas não são gravadas em arquivo nem exibidas no terminal: elas são escritas em um destino que descarta os dados (`std::io::sink`), com o mesmo código nos dois ambientes. Dessa forma, o tempo medido não inclui operações de entrada e saída, e a saída recebe o mesmo tratamento no Rust nativo e no WebAssembly.

O *n-body* registrou *overhead* de 69,7%. A simulação envolve cálculos iterativos de forças gravitacionais entre cinco corpos, com uso intensivo de operações de ponto flutuante de dupla precisão, incluindo raízes quadradas, sobre um conjunto reduzido

de dados. Esse resultado pode ser atribuído ao fato de que, no perfil padrão, o WebAssembly executa esses cálculos apenas de forma escalar, enquanto o código nativo pode usar instruções vetoriais, que processam dois valores de dupla precisão por operação.

O mandelbrot apresentou o maior *overhead* observado, correspondente a 129,9%. Nesse cenário, o tempo de execução em WebAssembly foi mais que o dobro do observado na execução nativa. O algoritmo realiza processamento intensivo de ponto flutuante de dupla precisão e grava o resultado em um *buffer* de *bits* mantido na memória, com cerca de 30,5 MiB. A imagem não é gravada em arquivo, ao final, apenas um checksum desse *buffer* é impresso com a macro (`println!`). No ambiente nativo, essa linha vai para a saída padrão do contêiner, no WebAssembly, é descartada, pois o alvo (`wasm32-unknown-unknown`) não possui saída padrão. Como se trata de uma única linha por execução, essa diferença é desprezível diante do tempo de processamento e não explica o *overhead* observado. Uma possível explicação para essa diferença de desempenho está na vetorização realizada pelo compilador na execução nativa, que permite processar dois valores de ponto flutuante de dupla precisão por instrução, enquanto, na implementação WebAssembly avaliada, esses valores são processados separadamente.

Considerando todos os algoritmos avaliados, o *overhead* médio foi de 47,7% (Equação (3.14)), enquanto a mediana foi de 18,1% (Equação (3.15)). A diferença entre essas métricas indica que os casos extremos, especialmente o mandelbrot, exercem influência significativa sobre a média geral dos resultados.

O *overhead* de alguns algoritmos oscilou entre os computadores, que possuem configurações de *hardware* e sistema operacional distintas, como no spectral-norm, entre 4,0% e 25,9%, e no n-body, entre 46,9% e 94,4%. Ainda assim, a separação entre os grupos se manteve em todas as máquinas: mandelbrot e n-body registraram os maiores *overheads* nos três computadores, enquanto fannkuch-redux, spectral-norm e fasta permaneceram abaixo de 26%. Dentro de cada computador, a variação entre as 10 execuções foi pequena: o coeficiente de variação, razão entre o desvio padrão e a média, ficou em até 3,2% na maioria dos casos. Os maiores valores ocorreram no WebAssembly, no n-body e no spectral-norm do computador 2, com 8,2% e 9,5%, e no fannkuch-redux do computador 3, com 7,0%, casos em que o uso da mediana reduz a influência das execuções atípicas.

Tabela 2 – Tempo mediano e desvio padrão das execuções por computador

Algoritmo	Computador	Rust (ms)		Wasm (ms)	
		Mediana	Desvio padrão	Mediana	Desvio padrão
fannkuch-redux	1	19.241,4	173,9	19.884,9	247,2
	2	21.099,7	73,5	22.865,5	73,6
	3	14.931,1	164,7	16.181,8	1.176,6
spectral-norm	1	1.017,0	12,4	1.151,9	5,0
	2	1.304,8	28,1	1.356,7	133,9
	3	902,7	6,2	1.136,5	7,6
fasta	1	1.504,4	6,5	1.704,9	44,6
	2	1.688,1	12,2	2.070,0	17,6
	3	1.283,9	41,5	1.522,6	29,9
n-body	1	1.464,0	4,7	2.504,1	8,6
	2	1.765,7	22,8	3.431,7	293,4
	3	1.661,9	29,4	2.441,3	50,4
mandelbrot	1	2.829,7	4,5	6.143,2	73,0
	2	3.429,1	23,7	8.172,5	47,6
	3	1.981,1	9,2	4.655,2	59,1

Fonte: Os autores

4.2 USO DE MEMÓRIA

As métricas de *baseline*, pico e consumo líquido foram calculadas pelas Equações (3.10) a (3.12) para cada computador e consolidadas pela média aritmética entre eles (Equação (3.13)). As Tabelas 3 e 4 apresentam os resultados obtidos.

Tabela 3 – Consumo de memória do executável Rust (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	14,0	15,0	1,0
n-body	9,6	10,9	1,2
spectral-norm	8,2	9,2	0,9
mandelbrot	8,2	40,5	32,3
fasta	8,7	9,7	1,0

Fonte: Os autores

Tabela 4 – Consumo de memória do executável WebAssembly (MiB)

Algoritmo	Base	Pico	Líquido
fannkuch-redux	233,3	251,6	18,3
n-body	211,8	215,1	3,3
spectral-norm	209,3	211,2	1,9
mandelbrot	209,9	246,1	36,2
fasta	211,1	213,9	2,8

Fonte: Os autores

O *baseline* do ambiente Rust situou-se entre 8,2 MiB e 14,0 MiB, refletindo principalmente o consumo do contêiner em estado ocioso. O valor mais alto, observado no fannkuch-redux, decorre do computador 2, que registrou 25,0 MiB nessa execução, enquanto os demais ficaram próximos de 8 MiB. No ambiente WebAssembly, os valores de *baseline* variaram entre 209,3 MiB e 233,3 MiB, em razão da execução permanente do Chrome Headless e do Puppeteer, e o valor mais alto, também no fannkuch-redux, decorre do mesmo computador, com 273,0 MiB.

Dessa forma, a comparação direta entre os valores totais de memória não é adequada para avaliar exclusivamente o impacto dos algoritmos. Por esse motivo, a análise foi concentrada no consumo líquido de memória.

Considerando essa métrica, o Rust apresentou consumo líquido entre 0,9 MiB e 1,2 MiB em quatro dos cinco algoritmos: fannkuch-redux, n-body, spectral-norm e fasta. A exceção foi o mandelbrot, com 32,3 MiB, valor praticamente idêntico nos três computadores e compatível com o *buffer* alocado pelo algoritmo, que usa um *bit* por *pixel* de uma imagem de 16.000 por 16.000 *pixels*, totalizando cerca de 30,5 MiB.

No WebAssembly, o consumo líquido ficou entre 1,9 MiB e 3,3 MiB para n-body, spectral-norm e fasta, ligeiramente acima dos valores do Rust nativo. O mandelbrot também apresentou o maior consumo nesse ambiente, com 36,2 MiB, valor igualmente compatível com o *buffer* de saída, que no WebAssembly é alocado na memória linear do módulo. O fannkuch-redux registrou 18,3 MiB, porém com grande variação entre os computadores, de 5,6 MiB a 38,4 MiB.

De modo geral, o consumo líquido do WebAssembly foi maior que o do Rust nativo em todos os algoritmos.

4.3 SÍNTESE COMPARATIVA

Os resultados mostram que o Rust nativo foi mais rápido em todos os algoritmos e que o *overhead* do WebAssembly variou de forma expressiva conforme o algoritmo. Fannkuch-redux, spectral-norm e fasta ficaram entre 6,7% e 18,1%, enquanto n-body e mandelbrot chegaram a 69,7% e 129,9%, separação que se manteve nos três computadores.

Também o ambiente WebAssembly exige, em repouso, entre 201,1 MiB e 219,3 MiB a mais que o ambiente nativo, com *baseline* entre 209,3 MiB e 233,3 MiB, contra 8,2 MiB a 14,0 MiB no Rust, para manter o Chrome Headless através do Puppeteer em execução, valor que variou pouco entre os algoritmos e independe da carga processada, tornando a comparação direta com o ambiente nativo inadequada. Por esse motivo, a análise de consumo de memória foi concentrada no consumo líquido, que isolou o impacto do algoritmo em execução. Por essa métrica, os dois ambientes apresentaram consumo reduzido na maioria dos algoritmos, com o WebAssembly

ligeiramente acima do Rust nativo, e o maior consumo em ambos foi o do mandelbrot, determinado pelo tamanho do *buffer* de saída.

4.4 DIRETRIZES PARA ADOÇÃO DO WEBASSEMBLY

Com base na síntese dos resultados experimentais, foram elaboradas diretrizes técnicas para orientar a tomada de decisão sobre em quais cenários a execução de código Rust via WebAssembly representa uma alternativa viável à execução nativa. O Quadro 1 sintetiza a classificação das cargas computacionais avaliadas, o *overhead* consolidado de tempo observado em relação à execução nativa e a recomendação técnica de viabilidade correspondente.

Quadro 1 – Diretrizes de viabilidade do Wasm conforme a carga

Tipo de Carga	Algoritmo	Overhead (%)	Viabilidade do Wasm
Operações com inteiros, manipulação de arrays	fannkuch-redux	6,7%	Altamente viável, baixo custo adicional.
Laços de repetição com ponto flutuante	spectral-norm	14,0%	Altamente viável, desempenho próximo ao nativo.
Geração de aleatórios e manipulação em memória	fasta	18,1%	Viável, desempenho aceitável.
Ponto flutuante iterativo e intensivo	n-body	69,7%	Inviável, mas exige testes com vetorização no Wasm.
Processamento numérico intensivo e manipulação de bits	mandelbrot	129,9%	Inviável, também exige experimentos com vetorização.

Fonte: Os autores

4.5 LIMITAÇÕES DO ESTUDO

Há limitações que devem ser consideradas neste estudo:

- Nos testes realizados, a variável de ambiente `RAYON_NUM_THREADS` foi fixada em 1, restringindo a execução de algoritmos que utilizam a biblioteca *Rayon* a uma única *thread*, mesmo naqueles casos em que o processamento paralelo poderia representar ganhos de desempenho. Essa restrição foi necessária em razão de erros encontrados na compilação da biblioteca *wasm-bindgen-rayon* para o ambiente WebAssembly, e foi replicada no ambiente Rust nativo para manter a padronização entre as comparações.

- Não foram utilizadas opções adicionais de compilação para o Rust além das configurações padrão do perfil `release`. Em particular, a flag `+simd128` não foi habilitada para o WebAssembly, seu uso poderia reduzir a diferença de desempenho entre o Rust nativo e o WebAssembly nos algoritmos mandelbrot e n-body.
- Estudos futuros poderiam empregar ambientes de execução WebAssembly independentes de navegador, como *Wasmer* e *Wasmtime*, permitindo isolar de forma mais precisa o desempenho do código WebAssembly.
- A avaliação do ambiente WebAssembly limitou-se ao navegador Google Chrome, controlado por meio do Puppeteer, o que restringe a generalização dos resultados obtidos. Diferentes navegadores utilizam motores de execução WebAssembly distintos, cada um com estratégias próprias de otimização, o que pode alterar significativamente os resultados de desempenho.

5 CONCLUSÃO

Este trabalho realizou uma análise comparativa entre a execução nativa de código Rust e sua execução compilada para WebAssembly, avaliando os impactos em tempo de execução e consumo de memória. A utilização de CPU também foi coletada, mas mostrou-se pouco informativa para a comparação, pelos motivos descritos na Seção 3.4.3, sendo utilizada apenas para identificar os períodos de execução.

Para embasar essa análise, foram abordados os fundamentos da linguagem Rust e as características do WebAssembly na revisão da literatura. Além disso, na etapa prática foram selecionados, do The Computer Language Benchmarks Game, 5 algoritmos de diferentes classes de problemas, implementados em Rust, portados para WebAssembly e executados em ambiente controlado via Docker em 3 máquinas distintas. Os resultados mostraram que o Rust nativo foi mais rápido que o WebAssembly em todos os algoritmos avaliados, com *overhead* do WebAssembly entre 6,7%, no *fannkuch-redux*, e 129,9%, no *mandelbrot*, e mediana de 18,1%. A análise de memória evidenciou que parte do consumo do ambiente WebAssembly não está associada à execução dos algoritmos, mas ao ambiente de execução, que mantém componentes como o Chrome Headless e o Puppeteer em funcionamento contínuo e eleva o consumo de memória em repouso para cerca de 210 MiB.

Diante disso, a utilização das métricas de *baseline* e consumo líquido mostrou-se fundamental para isolar o impacto real dos algoritmos, permitindo uma análise mais precisa e justa entre os ambientes. Por essa métrica, o consumo líquido dos dois ambientes foi reduzido na maioria dos algoritmos, com o WebAssembly ligeiramente acima do Rust nativo, e o maior consumo em ambos foi o do *mandelbrot*, determinado pelo tamanho do *buffer*. Observou-se ainda que o *overhead* de tempo variou de forma expressiva entre os algoritmos, com separação consistente nos três computadores.

Conclui-se, portanto, que o WebAssembly pode representar uma alternativa viável à execução nativa para aplicações Rust executadas no navegador, com custo adicional entre 6,7% e 18,1% em três dos cinco algoritmos avaliados. Em outras cargas, porém, o custo chegou a superar o dobro do tempo nativo, de modo que, quando o desempenho é o principal requisito, a adoção do WebAssembly deve ser precedida da medição com a carga de trabalho específica da aplicação.

Como limitação, os algoritmos foram executados com uma única *thread* nos dois ambientes, pois não foi possível habilitar o processamento paralelo no WebAssembly. Como trabalhos futuros, sugere-se avaliar a execução *multi-thread* no navegador, ampliar o conjunto de algoritmos e comparar outros mecanismos de execução de WebAssembly.

REFERÊNCIAS

- Bugden, W.; Alahmar, A. Rust: The programming language for safety and performance. **arXiv**, v. 2206.05503, 2022. Disponível em: <<https://arxiv.org/abs/2206.05503>>. Citado 5 vezes nas páginas 13, 14, 15, 19 e 20.
- Fleming, P. J.; Wallace, J. J. How not to lie with statistics: the correct way to summarize benchmark results. **Communications of the ACM**, v. 29, n. 3, p. 218–221, 1986. Citado na página 28.
- Fulgham, B.; Gouy, I. **The Computer Language Benchmarks Game**. 2026. Acesso em: 18 Jun. 2026. Disponível em: <<https://benchmarksgame-team.pages.debian.net/benchmarksgame/>>. Citado 3 vezes nas páginas 11, 22 e 23.
- Haas, A. *et al.* Bringing the web up to speed with webassembly. **Communications of the ACM**, v. 61, n. 12, p. 107–115, 2018. Disponível em: <<https://cacm.acm.org/research/bringing-the-web-up-to-speed-with-webassembly>>. Citado na página 15.
- Jangda, A. *et al.* Not so fast: Analyzing the performance of webassembly vs. native code. **Proceedings of the 2019 USENIX Annual Technical Conference (USENIX ATC '19)**, p. 107–120, 2019. Disponível em: <<https://www.usenix.org/conference/atc19/presentation/jangda>>. Citado 3 vezes nas páginas 12, 20 e 21.
- MDN Web Docs. **WebAssembly**. 2026. Acesso em: 18 jun. 2026. Disponível em: <<https://developer.mozilla.org/en-US/docs/WebAssembly>>. Citado na página 15.
- Moric, Z.; Branstett, L.; Petrunic, R. Rust and webassembly for fast, secure, and reliable software. **Proceedings of the 33rd DAAAM International Symposium on Intelligent Manufacturing and Automation**, DAAAM International, Vienna, Austria, v. 33, p. 165–171, 2022. Disponível em: <<https://doi.org/10.2507/33rd.daaam.proceedings.023>>. Citado 3 vezes nas páginas 16, 17 e 20.
- Thomas, G. **A Proactive Approach to More Secure Code**. 2019. Acesso em: 7 out. 2026. Disponível em: <<https://www.microsoft.com/en-us/msrc/blog/2019/07/a-proactive-approach-to-more-secure-code>>. Citado na página 13.
- World Wide Web Consortium (W3C). **WebAssembly Core Specification**. 2019. W3C Recommendation, 5 dez. 2019. Disponível em: <<https://www.w3.org/TR/2019/REC-wasm-core-1-20191205/>>. Citado na página 15.

APÊNDICE A – CONSUMO DE MEMÓRIA POR COMPUTADOR

Este apêndice apresenta as métricas de memória obtidas em cada um dos três computadores descritos na Seção 3.3, antes da consolidação pela média aritmética (Equação (3.13)). A Tabela A1 apresenta o *baseline*, o pico e o consumo líquido de cada algoritmo e ambiente (Equações (3.10) a (3.12)).

Tabela A1 – Consumo de memória por computador (MiB)

Algoritmo	Computador	Rust			Wasm		
		Base	Pico	Líquido	Base	Pico	Líquido
fannkuch-redux	1	8,0	9,0	1,0	222,1	227,7	5,6
	2	25,0	26,2	1,2	273,0	311,4	38,4
	3	8,9	9,6	0,7	204,9	215,7	10,8
spectral-norm	1	8,2	9,1	0,9	222,1	224,3	2,1
	2	8,3	9,3	1,0	211,0	213,1	2,1
	3	8,1	9,1	1,0	194,8	196,3	1,5
fasta	1	8,0	9,0	0,9	223,2	225,8	2,6
	2	10,2	11,2	1,0	214,9	218,5	3,6
	3	7,9	8,9	1,0	195,2	197,4	2,2
n-body	1	8,0	9,1	1,1	222,8	225,2	2,4
	2	13,1	14,2	1,1	217,9	221,6	3,7
	3	7,8	9,2	1,4	194,6	198,5	3,9
mandelbrot	1	8,3	40,4	32,1	222,5	258,6	36,1
	2	8,5	40,7	32,2	211,0	246,8	35,8
	3	7,8	40,4	32,5	196,1	232,9	36,8

Fonte: Os autores